

SWE308: Information Security
Lecture 8: Symmetric Cryptography - Integrity Assurance
If $MAC = MAC'$, then M is authentic; else discard M

Dr Iftekhar Salam
iftekhar.salam@xmu.edu.my

Xiamen Univesity Malaysia

Presentation Outline

- 1 Cryptography and Integrity Assurance
- 2 Hash Functions based Integrity Assurance
- 3 Block Cipher Modes for Integrity Assurance
- 4 Authenticated Encryption

Presentation Outline

- 1 Cryptography and Integrity Assurance
- 2 Hash Functions based Integrity Assurance
- 3 Block Cipher Modes for Integrity Assurance
- 4 Authenticated Encryption

Secrecy vs Integrity

- So far we have been concerned with ensuring *secrecy*, i.e., "confidentiality" of communication
- What about integrity
 - ensuring that a received message originated from the intended party, and was not modified
- **Passive attacks vs. active attacks**
 - So far we have been considered only passive (i.e., eavesdropping) attack—attacker simply observes the channel
 - In the setting of integrity, we explicitly consider active attacks—attacker has full control over the channel
- **Secrecy and integrity**
 - Possible to have either one without the other
 - Sometimes you might want one without the other
 - Most often, both are needed

What is Integrity Assurance?

Detecting the **unauthorised** modification of information

- Alice sends a message m to Bob
- Eve modifies the message to m'
- Bob has a mechanism to verify whether the received message m' is modified or not
 - **Note that this mechanism does not prevent Eve from modifying the message**
 - What type of control measure is this?

How Can You Be Sure About The Integrity of Information

In storage:

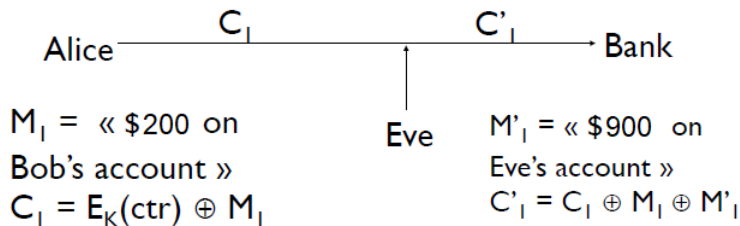
- Q: Is the file you have stored the same as it was when you stored it, or has it been altered
 - Accidentally?
 - Deliberately?

During transmission:

- Q: Is the information being sent over network communication links likely to be changed in transit
 - Is the message sent the same as the message received or has it been altered accidentally?
 - Is the message sent the same as the message received or has it been altered deliberately?

Encryption Does Not Provide Integrity

For example, CTR mode ensures confidentiality if the block cipher used is secure. However, no integrity is guaranteed.



Similarly, an adversary can forge the first block of CBC mode

- Find out WHY?

Error Detection (and Correction) Codes Do Not Provide Integrity

For example, consider Cyclic Redundancy Code (CRC)

- A method to detect random errors
- Do not protect against a malicious adversary who can choose exactly where to introduce an arbitrary number of changes
 - An active attacker can easily modify message and re-compute CRC

Integrity Assurance using Crypto?

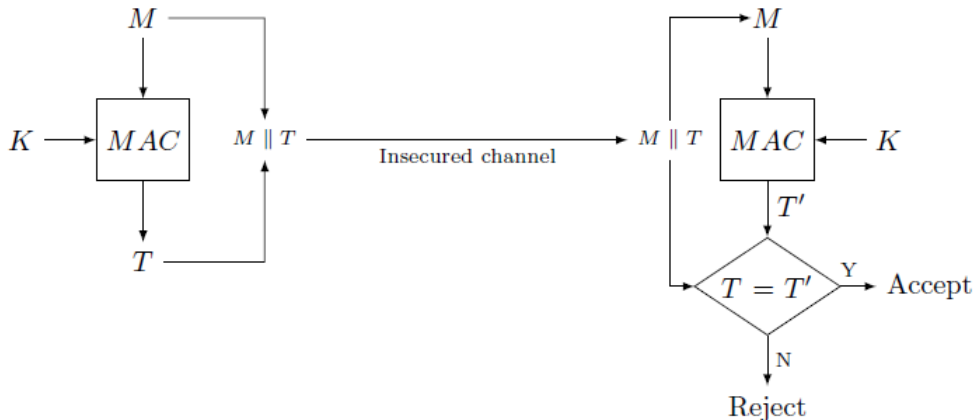
So, till now we were talking about the security service: confidentiality?

Can we use cryptographic techniques to provide integrity assurance?

- Recall that integrity assurance detects the unauthorized modification of data
- The answer is: Yes, we can use cryptographic techniques to provide integrity assurance
 - hash function
 - block ciphers
 - stream ciphers
 - Also, public key/asymmetric algorithms can be used to provide integrity assurance

Symmetric Crypto to Provide Integrity Assurance

We can use Message Authentication Code (MAC) algorithms to provide integrity assurance



Source: eprints.qut.edu.au/119361/1/Hassan%20Musallam%20Ahmed_Qahur%20A1%20Mahri_Thesis.pdf

Requirements of A MAC Algorithm

MAC algorithms need the following properties:

- Ease of computation: it is easy to compute the tag T for a given message M and secret key K
- Compression: the MAC scheme digests an arbitrary length message to produce a fixed-length value.

For a MAC algorithm to be secure, it also need to satisfy the following properties:

- If the MAC tag T is known, it is difficult to find a message M such that $MAC(K; M) = T$
- It is practically infeasible to compute a valid pair $(M_j; T_j)$ even if the attacker is given a set of pairs $(M_i; T_i)$ such that $M_j \neq M_i$ in all given pairs.

Q: Seems similar to anything that we studied earlier in this semester?

Attack against MAC

MAC Forgery

- Adversary tries to create a valid MAC tag for a message that the sender never actually sent
- A MAC forgery attack is successful if the attacker can forge a correct MAC with a probability better than trying all the possible MAC values

Two types:

- Existential forgery: An attacker can forge a MAC but need not have any control on the corresponding message contents
- Selective forgery: The attacker can forge a MAC tag for a message of their choice

Primitives to Construct A MAC Tag

Commonly used methods to provide assurances of message integrity

- Hash functions
 - Often used in information security applications
 - **BUT** dedicated hash functions (also known as simple hash functions) don't use a secret key - need to understand appropriate use and limitations
 - Instead of dedicated hash functions, one can use cryptographic hash functions which involves a secret key
- Certain modes of block ciphers provide integrity assurance
- Stream ciphers can also be used for integrity assurance
 - **BUT** this is a newer and not yet fully explored research area

Presentation Outline

- 1 Cryptography and Integrity Assurance
- 2 Hash Functions based Integrity Assurance
- 3 Block Cipher Modes for Integrity Assurance
- 4 Authenticated Encryption

Review from Lecture 3: Basic Properties of hash Function

Important Hash function properties:

- Fixed length output $H(M)$, for arbitrary length input M
- Small changes in M will make major changes in $H(M)$
- It is easy to compute $H(M)$ from M , but given $H(M)$, it is infeasible to compute M . A hash function with this property is referred to as **one-way or preimage resistant**
- For any M and $H(M)$, it is computationally infeasible to find $M \neq M'$ with $H(M) = H(M')$. A hash function with this property is referred to as **second preimage resistant**. This is sometimes referred to as **weak collision resistant**.
- For any given $H(M)$, it is hard/infeasible to find M and M' with $H(M) = H(M')$. A hash function with this property is referred to as **collision resistant**. This is sometimes referred to as **strong collision resistant**.

Watch: <https://www.youtube.com/watch?v=sGcMfKlw9n8>

Dedicated Hash Function for Message Integrity Assurance

- ① Alice and Bob agrees on what hash function $H()$ to use
- ② Alice (sender) performs the following:
 - ① Alice generates message M
 - ② Alice computes the hash value $H(M)$
 - ③ Alice sends both M and $H(M)$

Dedicated Hash Function for Message Integrity Assurance

- ① Alice and Bob agrees on what hash function $H()$ to use
- ② Alice (sender) performs the following:
 - ① Alice generates message M
 - ② Alice computes the hash value $H(M)$
 - ③ Alice sends both M and $H(M)$
- ③ Bob (receiver) receives M' and $H(M)$
 - ① Bob uses M' to compute the hash value $H(M')$
 - ② Bob compares $H(M)$ and $H(M')$
 - If $H(M) = H(M')$, then Bob assumes that the received message is the same as the one that Alice sent
 - If $H(M) \neq H(M')$, then Bob knows that the received message is not the same as the one that Alice sent

Dedicated Hash Function: Application Examples

Use hash value as a manipulation detection code (MDC) for data. MDC used as a checksum to detect if data has been modified

Example: Password verification

- On a restricted system, store the hashes (or salted hashes) of user passwords, rather than the password itself
- When a user logs in, compute the hash (or salted hash) of the password they submit and compare with the stored hash
- If they match, assume password provided for that user is correct

Example: File verification

- Compare the hash of a local binary file to the hash of the official binary file to verify the integrity of your local file
- Example: Microsoft's File Checksum Integrity Verifier
 - Computes MD5 or SHA1 hashes

Problems with Simple Hash Functions to provide Integrity Assurance?

Integrity assurance using dedicated/simple hash function:

- Useful for detecting accidental errors/modifications
- **NOT useful** in detecting deliberate/intentional modifications (active attack)

Active attacker can perform a MITM attack

Problems with Simple Hash Functions to provide Integrity Assurance?

Integrity assurance using dedicated/simple hash function:

- Useful for detecting accidental errors/modifications
- **NOT useful** in detecting deliberate/intentional modifications (active attack)

Active attacker can perform a MITM attack

- 1 Intercepts the transmission $M, H(M)$ sent by Alice
- 2 Changes M to M''
- 3 Recalculate the hash of $H(M'')$
- 4 Sends $M'', H(M'')$ to Bob

If Bob relies on $H(M)$ for integrity assurance he will mistakenly think M'' sent by MITM is the unaltered message from Alice, which actually is not the case

Note: Mechanisms cannot prevent unauthorised modification of M

Better Solutions for Providing Integrity Assurance?

Use a keyed hash function

- Also called cryptographic hash function or message authentication code (MAC)
- A keyed hash function $H_K(M)$ has two inputs:
 - Message M
 - A cryptographic key K
- Using a secret key K protects against unauthorized modification of the hash value
 - Calculating $H_K(M)$ value requires knowledge of secret key
 - Attacker can only successfully modify information and update the $H_K(M)$ value appropriately if they know the key

Keyed Hash Function

Alice (sender) performs the following:

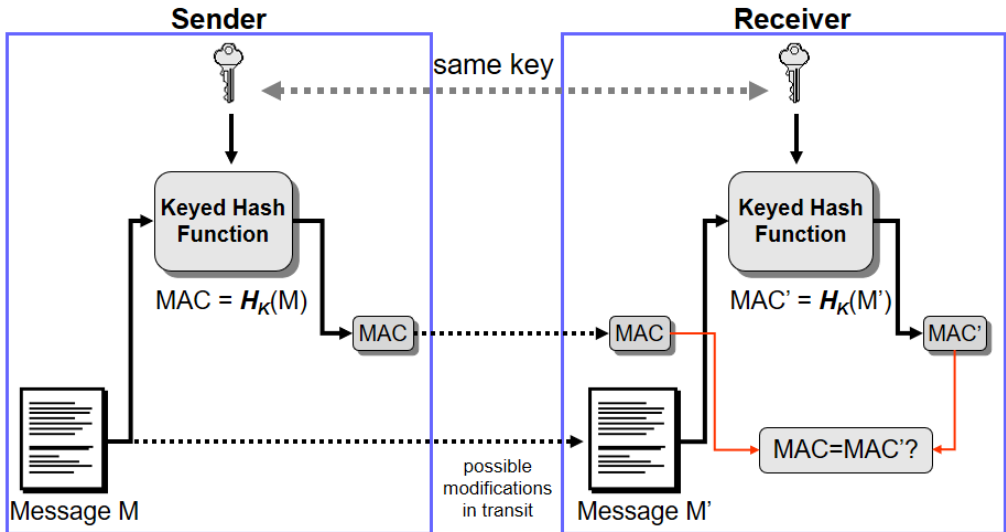
- 1 Alice generates message M
- 2 Alice computes the message authentication code: $MAC = H_K(M)$, using a pre-shared secret key K
- 3 Alice sends both M and MAC

Bob (receiver) receives M' and MAC

- 1 Bob uses M' to compute the message authentication code at the receiver:
 - $MAC' = H_K(M')$
- 2 Bob compares MAC and MAC'
 - If $MAC = MAC'$, then Bob assumes that the received message is the same as the one that Alice sent
 - If $MAC \neq MAC'$, then Bob knows that the received message is not the same as the one that Alice sent

Q: What Eve (adversary) needs to be able to compute the MAC?

Integrity Assurance using Keyed Hash Function



Keyed Hash Functions

Keyed hash functions can be built from regular hash functions, by including a secret key in a certain way

- For example, RFC 2104 describes HMAC: Keyed-Hashing for Message Authentication including
 - HMAC-MD5 based on the hash function MD5
 - HMAC-SHA1 based on the hash function SHA-1

Presentation Outline

- 1 Cryptography and Integrity Assurance
- 2 Hash Functions based Integrity Assurance
- 3 Block Cipher Modes for Integrity Assurance
- 4 Authenticated Encryption

Block Ciphers and Integrity Assurance

Certain block cipher modes can be used to provide integrity assurance

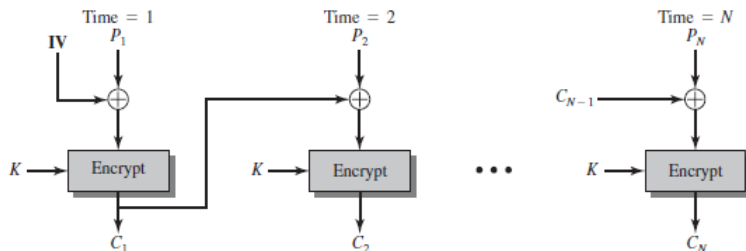
- Cipher Block Chaining Message Authentication Code (CBC-MAC)
- Counter with Cipher Block Chaining Message Authentication Code (CCM)
- Offset Codebook Mode (OCB)
- Galois/Counter Mode (GCM)
- ...

We will look into CBC-MAC and CCM

Remember Cipher Block Chaining(CBC)?

Cipher Block Chaining (CBC) mode encryption

- Plaintext data is divided into blocks $P_1, P_2, \dots, P_n$ (last block may require padding)
- The encryption algorithm now takes three inputs:
 - the key, the plaintext block and the previous ciphertext block
 - For the first plaintext block, an initialisation vector (IV) is used to ensure that two encryptions of the same plaintext will result in different ciphertext
- Encryption:**
 - $C_1 = E(K, [P_1 \oplus IV])$
 - $C_i = E(K, [P_i \oplus C_{i-1}])$



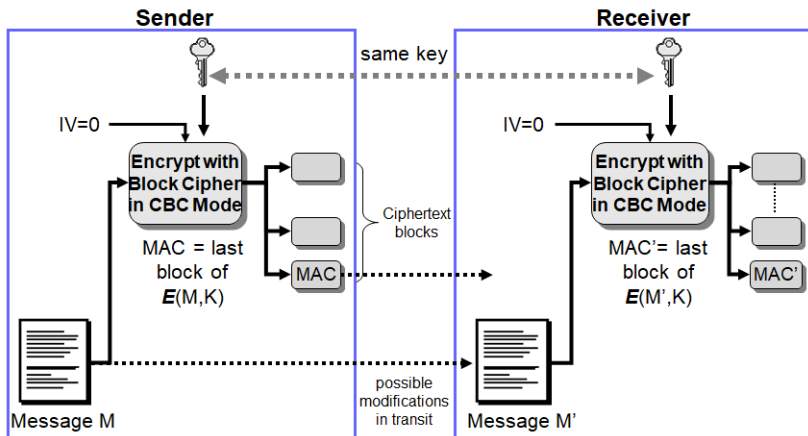
CBC-MAC is based on **CBC** mode, but provides integrity assurance rather than confidentiality.

Cipher Block Chaining Message Authentication Code (CBC-MAC)

A block cipher in CBC mode to provide integrity assurance, rather than confidentiality

- The file is encrypted using the block cipher in CBC mode
- IV is fixed (set to zero)
- The last ciphertext block is used as a Message Authentication Code (MAC) value
- Both the message and the MAC are sent to the receiver

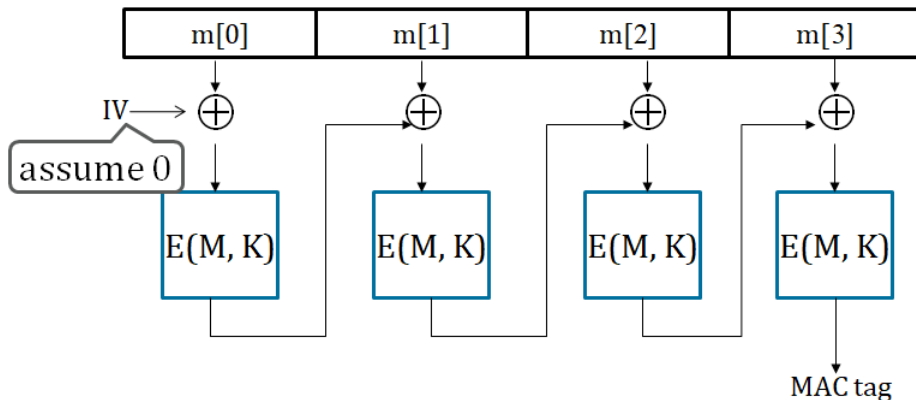
Considered secure for messages of a pre-selected fixed length



Cipher Block Chaining Message Authentication Code (CBC-MAC)

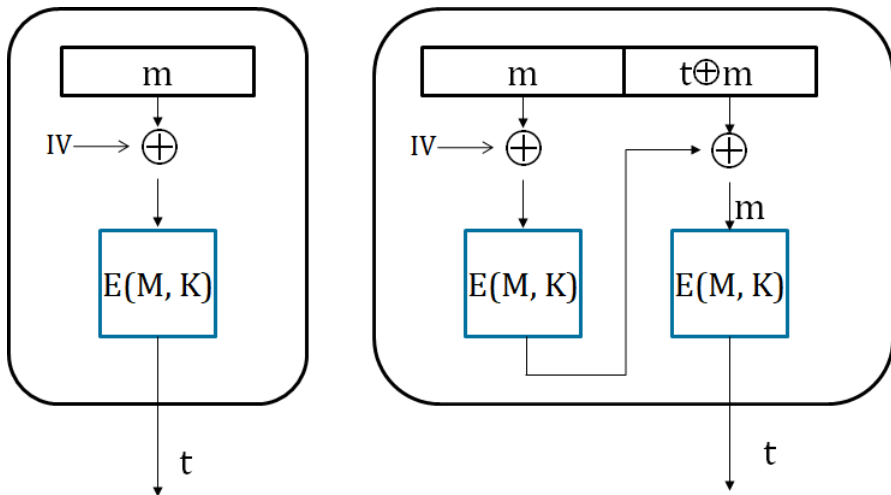
CBC-MAC

- IV is set to zero (**WHY?**)
- Sender encrypts message M using block cipher in CBC mode, with secret key K
- MAC = last ciphertext block (discard the other ciphertext blocks!)
- **Does CBC-MAC provide confidentiality?**
 - Think about what the sender sends!
- How the receiver will verify the integrity of the received message?



Why CBC-MAC is NOT Secure against Variable Length Messages?

Can break in 1-Chosen Message Attack

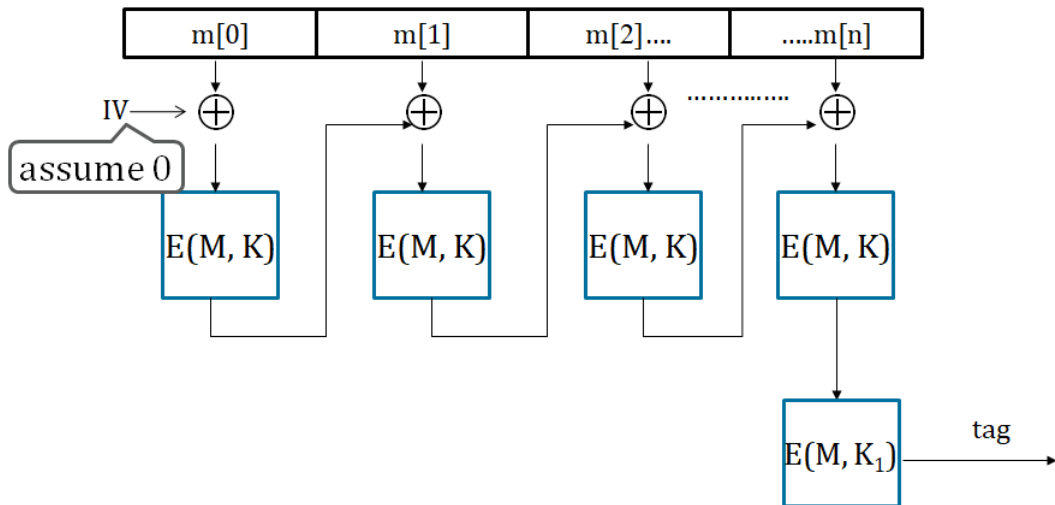


So, now we can create two different messages m and $m || t \oplus m$, and both of these messages have the same tag t

Encrypted CBC-MAC (ECBC-MAC)

Can we modify CBC-MAC so that it can be used with variable length messages?

- Encrypt the last block of output with a different key K_1



Presentation Outline

- 1 Cryptography and Integrity Assurance
- 2 Hash Functions based Integrity Assurance
- 3 Block Cipher Modes for Integrity Assurance
- 4 Authenticated Encryption

What is Authenticated Encryption (AE)?

What about if we want to provide both confidentiality and integrity assurance? Can we do that using a symmetric cipher?

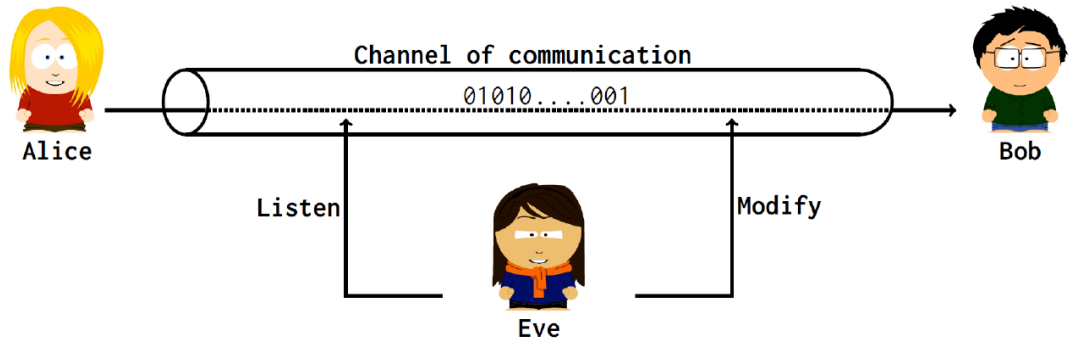


Figure: Problems Associated with Communication Over Insecure Channel

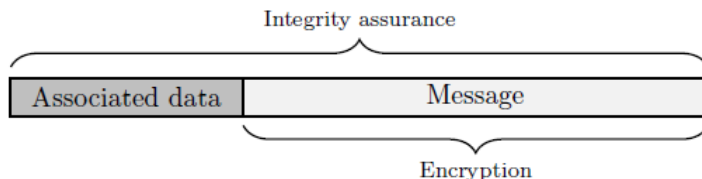
AE: Mechanism to provide secure communication in presence of adversary

- AE: Provide simultaneous confidentiality and integrity assurance
- Authenticated Encryption with Associated Data (AEAD)
 - Plaintext is authenticated & encrypted, and
 - Associated Data (AD) is authenticated

What is Associated Data (AD)?

Associated Data

- Not all data in applications are required to be encrypted, such as packet headers in internet protocols
- These headers should be in plaintext in order to be identified by different communicating parties
- However, these data still require integrity assurance to prevent attacks changing such headers
- Associated data is part of the message which does not require encryption BUT requires integrity assurance, e.g., the source or destination address



Source: eprints.qut.edu.au/119361/1/Hassan%20Musallam%20Ahmed_Qahur%20Al%20Mahri_Thesis.pdf

How to Construct Authenticated Encryption (AE)

Generic construction: Is it possible to combine confidentiality and integrity algorithms?

- YES! Generic composition was described by Bellare and Namprempre.
- Confidentiality component: Compute ciphertext C by encrypting the plaintext P .
 - Use a secure encryption algorithm
- Integrity component: Compute a message authentication code (MAC) tag T for the message M (Message can be either plaintext or ciphertext).
 - Use a Secure MAC algorithm
- Send C and T over the insecure channel.
- Receiver computes T' of the received message M' .
 - Valid message if $T = T'$
 - Decrypt message

Primitives for AE

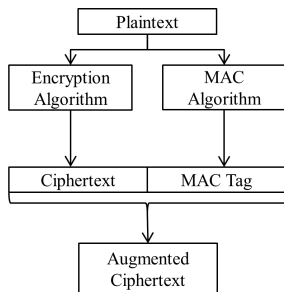
- Block ciphers, Stream ciphers, Sponge constructions, MAC and Hash functions

<https://eprint.iacr.org/2000/025.pdf>

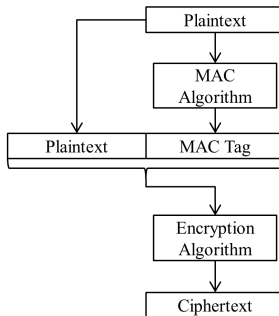
Generic AE Constructions

Order of encryption and authentication

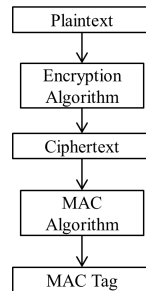
- Encrypt-and-MAC (E&M)
- MAC-then-Encrypt (MtE)
- Encrypt-then-MAC (EtM)



(a) Encrypt & MAC



(b) MAC-then-Encrypt



(c) Encrypt-then-MAC

Source: https://eprints.qut.edu.au/117673/1/Iftekhara_Salam_Thesis.pdf

Block Cipher Modes for AE

We can use certain block cipher modes for providing AE

- Counter with Cipher Block Chaining Message Authentication Code (CCM – (used in 802.11i))
- Offset Codebook Mode (OCB)
- Galois/Counter Mode (GCM)

We will have a look into the CCM mode

Counter with CBC MAC (CCM)

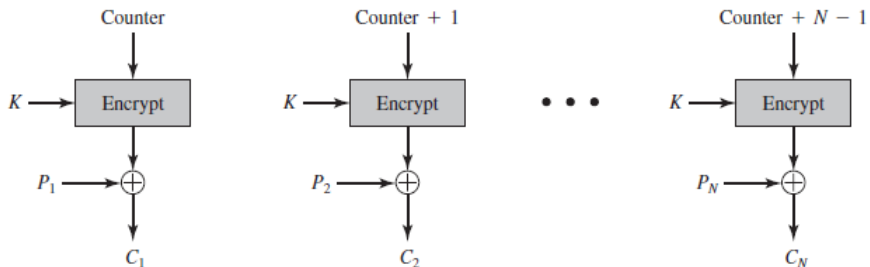
An authenticated encryption (AE) solution

- Encryption: Use of the block cipher in Counter (CTR) mode
- Message authentication: MAC computed with CBC-MAC
- Use the same key for encryption and message authentication algorithms
- CCM is used in Wi-Fi Protected Access II (WPA2), the successor to Wired Equivalent Privacy (WEP)
- CCM is standardised for use in IPsec and TLS1.2.

What is Counter Mode?

Counter Mode Encryption

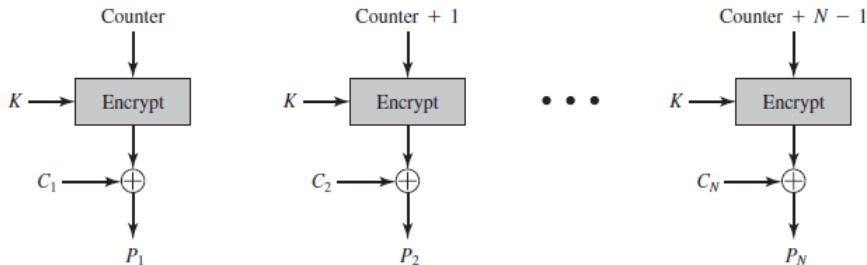
- A counter is equal to the plaintext block size
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block
- The counter is encrypted and then XORed with the plaintext block to produce the ciphertext block



What is Counter Mode?

Counter Mode Decryption

- The same sequence of counter values is used
- The counter is encrypted and then XORed with the ciphertext block to produce the plaintext block



CCM: MAC Generation and Encryption

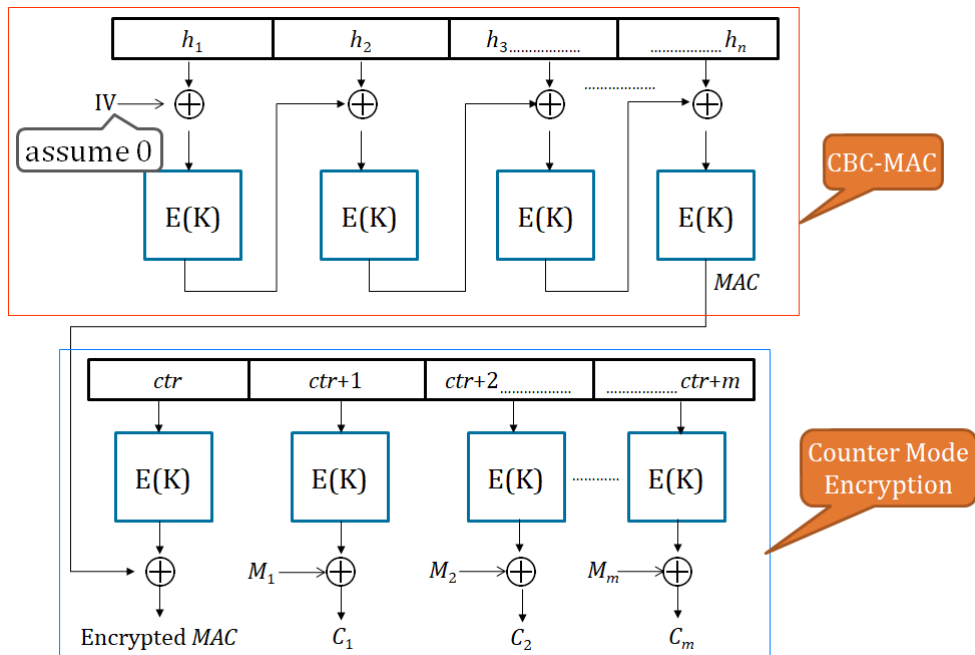
MAC Generation using CBC-MAC

- Use the CBC-MAC mode
- Construct a string $h = N || \text{len}(M)_{64} || M || \text{len}(AD)_{64} || AD || \text{Padding}$
- N is a nonce, M is message, AD is associated data
- $\text{len}(X)$ defines the length of X
- $\text{len}(X)_{64}$ means the 64-bit encoding of the length of string X
- Apply CBC-MAC to the string h
 - Outputs the MAC

Encryption using CTR mode

- Input to the encryption algorithm: $MAC || M$, where $M = M_1 || M_2 || \dots || M_m$
- Apply counter mode to the input to compute the ciphertext $C = C_1 || C_2 || \dots || C_m$
- Note that this will also output the **Encrypted MAC**

CCM: MAC Generation and Encryption



CCM: MAC Verification and Decryption

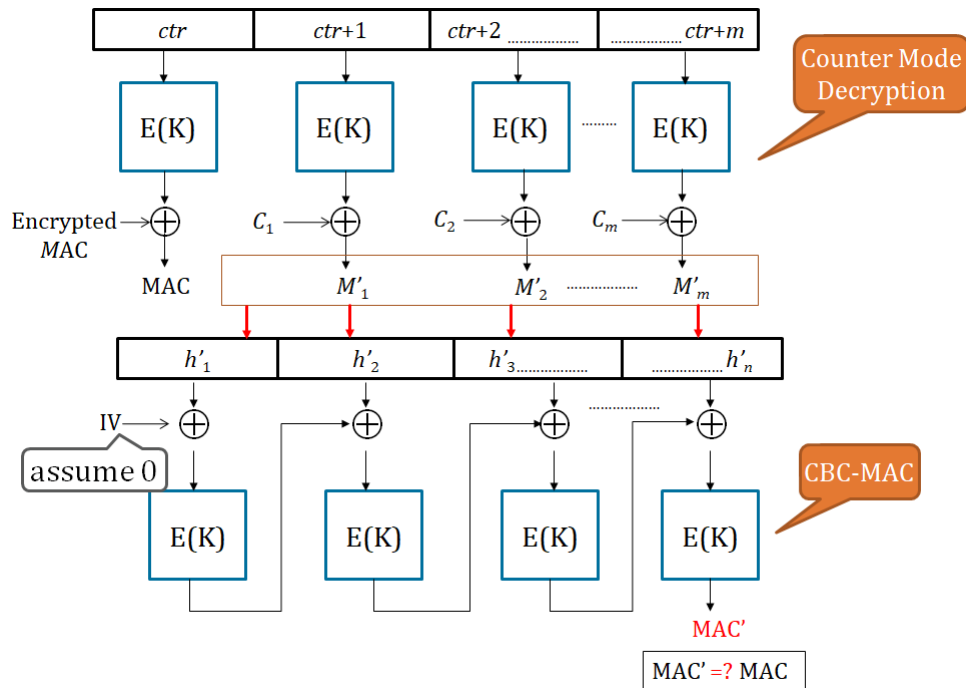
Decryption using CTR mode

- Recompute the key stream using **CTR mode** and XOR with the received message to decrypt the message $M' = M'_1 || M'_2 || \dots || M'_m$ and the decrypted MAC

MAC Verification

- The decrypted message M' is then used to reconstruct the string h'
- Apply CBC-MAC to h' to recompute the CBC-MAC value MAC'
- If the received MAC is equal to the computed MAC' at the receiver, then the message is verified
- Otherwise, the message is not verified, and the receiver **MUST NOT** reveal the decrypted message, the MAC, or any other information

CCM: MAC Verification and Decryption



Symmetric Ciphers and Security

Last two week's lecture we have seen the use of symmetric crypto to provide confidentiality. We can also use symmetric crypto to provide integrity assurance.

Integrity Assurance and Message Authentication

- HMAC and CBC-MAC use shared secret keys
- For message transmission scenarios, these MACs provide an additional security service of Message Authentication:
 - correct MAC value confirms the sender of the message is in possession of the shared secret key
 - This provides the receiver with some assurance of the authenticity of the message sender.

Confidentiality and Integrity Assurance

- MAC algorithm alone can not provide both of these security goals
- Similarly, confidentiality algorithms alone can not provide both of these security goals
- Can use authenticated encryption (AE) algorithms to provide both of these
 - Can use block cipher modes such as CCM, OCB, GCM etc
 - Recently there has been some development of stream cipher based AE algorithms

NOTE: An assurance of message integrity is meaningless *without knowing who sent the message*

End of Lecture 8! 😊

Next Lecture

- Asymmetric Cryptography

Remember to complete your online quiz for this week

QUESTIONS?

Appendix: University's vision and mission

- ① Vision: Xiamen University Malaysia aspires to become a university with a distinct global outlook, featuring first-class teaching and research, and embracing cultural diversity.
- ② Mission: To nurture young talents with dignity and wisdom, turning them into fine citizens of the region who will contribute to the prosperity of the people and social progress of Malaysia, China and Southeast Asia.